

# Distributed Systems

## Slip Test 2

### Question (5 marks)

Consider the following simple method to enforce mutual exclusion: all sites are arranged in a logical ring fashion and a unique token circulates around the ring hopping from a site to another site. When a site needs to execute its CS, it waits for the token, grabs the token, executes the CS, and then dispatches the token to the next site on the ring. If a site does not need the token on its arrival, it immediately dispatches the token to the next site (in zero time).

Assume there are  $N$  sites, the message/token delay is  $T$ , and the CS execution time is  $E$ .

Compare the above algorithm with Lamport & Ricart Agrawala wrt response time, #msgs and sync delay

### Solution

#### Key Variables:

- $N$  = Number of sites in the system
- $T$  = Maximum message/token delay
- $E$  = Critical Section (CS) execution time

### Lamport's Algorithm (1 mark)

- **Message Complexity (Worst Case):**  $3(N - 1)$
- **Explanation:** To execute the critical section, a requesting site must communicate with all other sites in the system. The breakdown is as follows:
  1. The site sends  $N - 1$  **REQUEST** messages to all other sites.

2. The site waits to receive  $N - 1$  **REPLY** messages from all other sites.
3. After executing the CS, the site sends  $N - 1$  **RELEASE** messages to all other sites to remove its request from their queues.

**Total:**  $(N - 1) + (N - 1) + (N - 1) = 3(N - 1)$  messages.

## Ricart-Agrawala Algorithm (1 mark)

- **Message Complexity (Worst Case):**  $2(N - 1)$
- **Explanation:** This algorithm optimizes Lamport's approach by removing the explicit **RELEASE** message.
  1. The requesting site sends  $N - 1$  **REQUEST** messages to all other sites.
  2. The site waits for  $N - 1$  **REPLY** messages. Sites not currently interested or with lower-priority requests reply immediately. Sites currently in the CS or with higher-priority requests defer their reply until they finish their own CS. Because the deferred **REPLY** acts as the permission to enter, it implicitly serves the function of the release message.

**Total:**  $(N - 1) + (N - 1) = 2(N - 1)$  messages.

## Ring Algorithm (3 marks)

### Message Complexity (1 mark)

- **Value (Worst Case):**  $N$
- **Explanation:** Under a light system load, a site might request the CS exactly after it has passed the token to its neighbor.
  1. The token must traverse the remaining  $N - 1$  sites to return to the requesting site ( $N - 1$  hops/messages).
  2. After executing the CS, the site must dispatch the token to the next site on the logical ring (1 hop/message).

**Total:**  $(N - 1) + 1 = N$  messages.

Note : Based on the explanation  $N - 1$  has also been awarded full marks

## B. Response Time (1 mark)

- **Value (Worst Case):**  $(N - 1)(T + E) + E$
- **Explanation:** Response time is defined as the time interval a request waits for its CS execution to be *over* after the request is sent.
  1. **Wait Time:** If a site requests the token right after dispatching it, and all other  $N - 1$  sites also wish to execute their CS, the token is held at each site for execution time  $E$  and takes transit time  $T$ . This results in a total wait time of  $(N - 1)(T + E)$ .
  2. **Execution Time:** Once the token arrives, the site executes its own CS, taking time  $E$ .

**Total:** Wait Time + Execution Time =  $(N - 1)(T + E) + E$ .

Note : Response Time with and without execution time has been awarded full marks

## C. Synchronization Delay (1 mark)

- **Value:**  $T * (N - 1)$
- **Explanation:** Synchronization delay is the time required after a site leaves the CS before the next site can enter the CS. In a logical token ring, when a site finishes its CS, it immediately passes the token to the next site. The delay occurs when the Token has to traverse all the sites to reach the last site which is interested in acquiring CS. So,  $N - 1$  hops with time delay of  $T$  in each in worst case.